

WRO Engineering Journal

Team Information:

Team Name: SSTitans

Team ID: FE-316-01

School: School of Science and Technology

Category: WRO Future Engineers

Team Members: - Elliott Gee Hsien Zhi———— (Team Leader)
- Lai Hong Yu ————— (Coder)
- Tejeshwar Nandagopal ——— (Builder)



Contents

- Hardware Overview
- Drivetrain
- Components
- Strategy
- Code

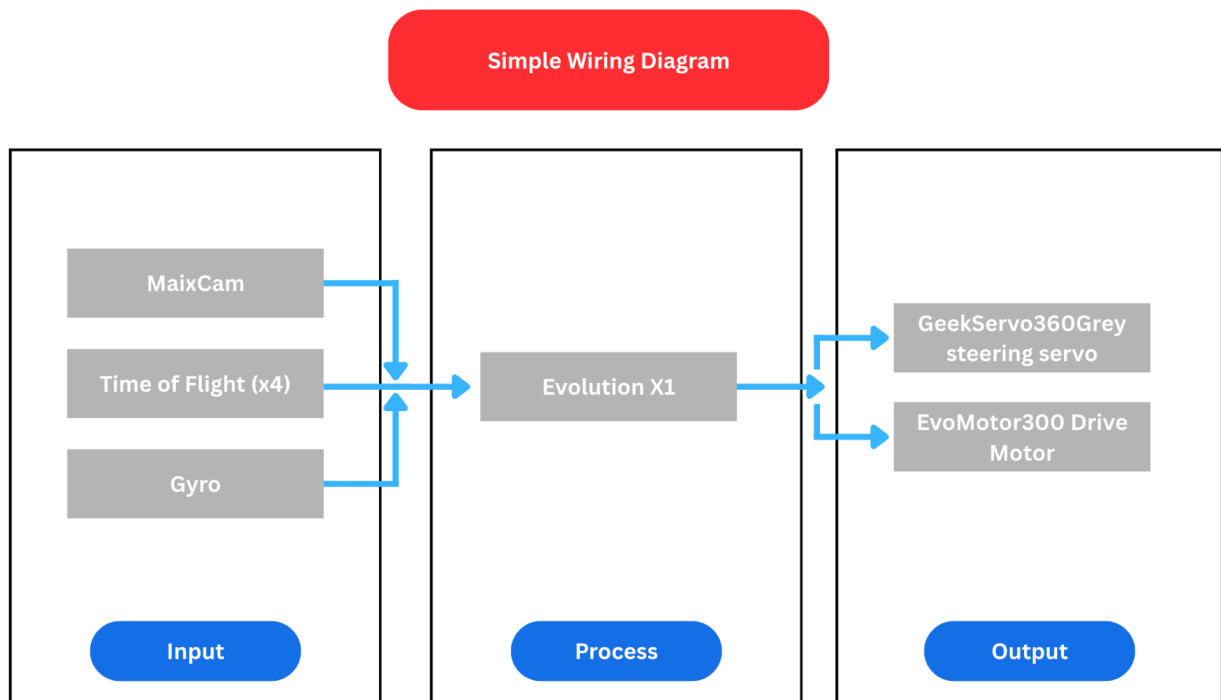
Hardware Overview

Table of Components used by the robot:

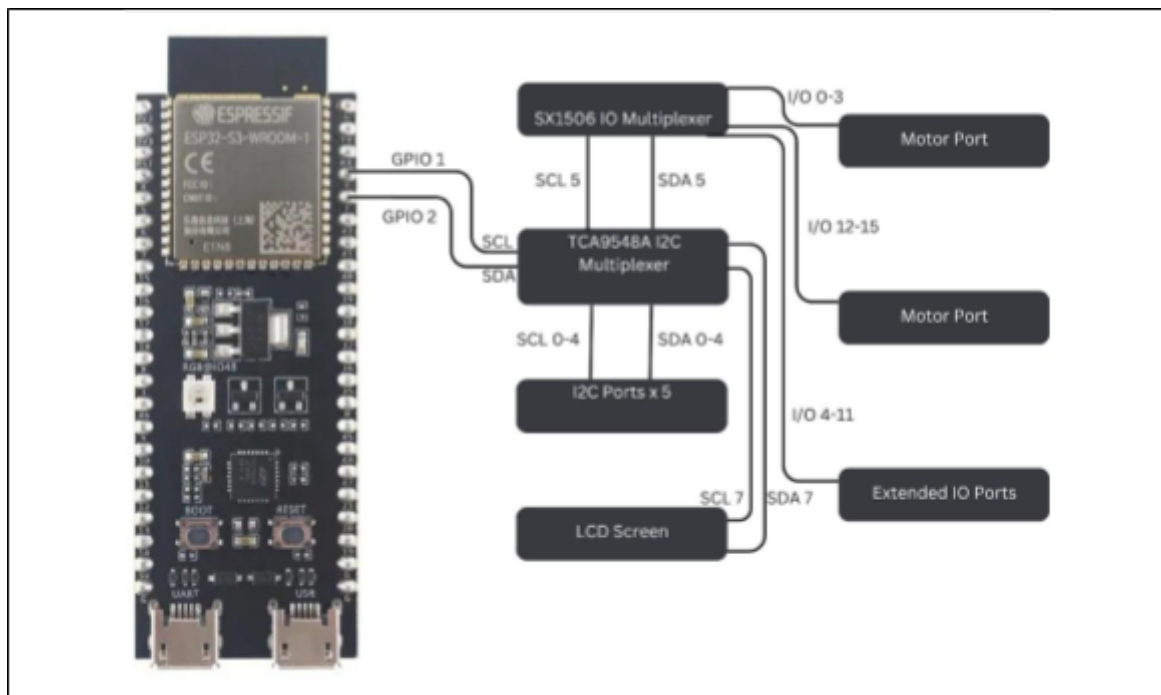
Part Name	Model	Quantity	Others
GeekServo	GeekServo360Grey	1	
EvoMotor	EvoMotor300	1	
Time of Flight	VL53L0X	4	
Camera	MaixCam	1	
Microcontroller	ESP32S3	1	
12C Multiplexer	TCA9548A	1	
IO Multiplexer	SX1506	1	
IMU	BNO055	1	
Battery	NCR18650B	2	

Diagrams

Simple Wiring Diagram (Blocks)



Microcontroller System Diagram



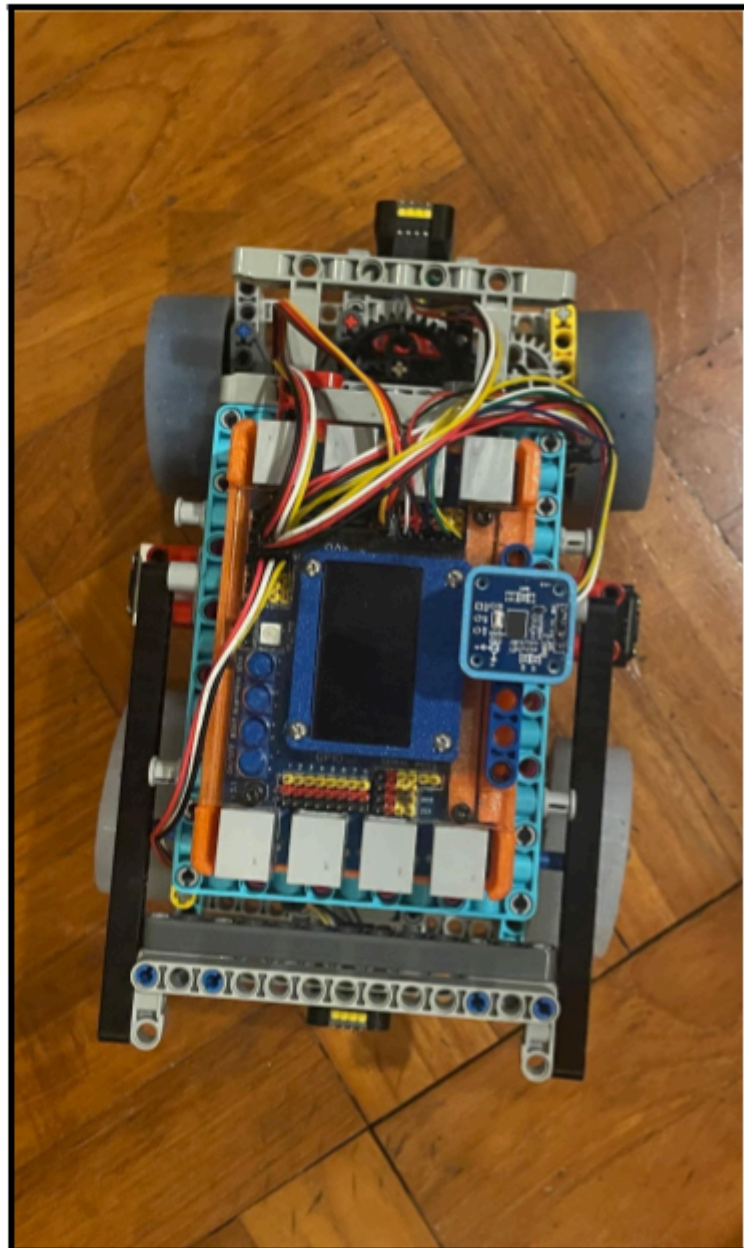
Drivetrain

Chassis Design

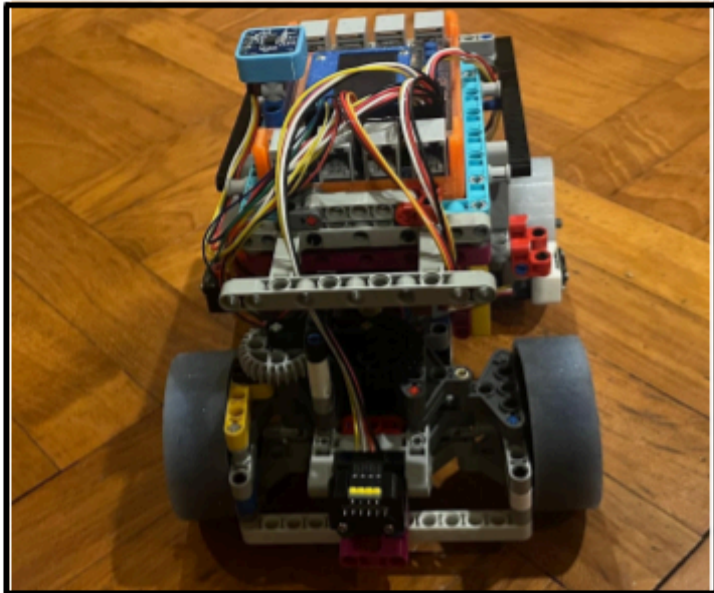
To build our chassis, we used LEGO parts, as they were light yet able to withstand the weight of the EvolutionX1 Brick. LEGO parts made it easy for us to prototype our models, as we built numerous prototype chassis before landing on the final design. We also made use of 3D-modelled holders for our sensors with LEGO compatibility; thus, it is easy to install and replace. We also designed it with a tight turning circle in mind, and as such we chose a 4WD & 4-wheel steering chassis.

Here are some photos of the vehicle:

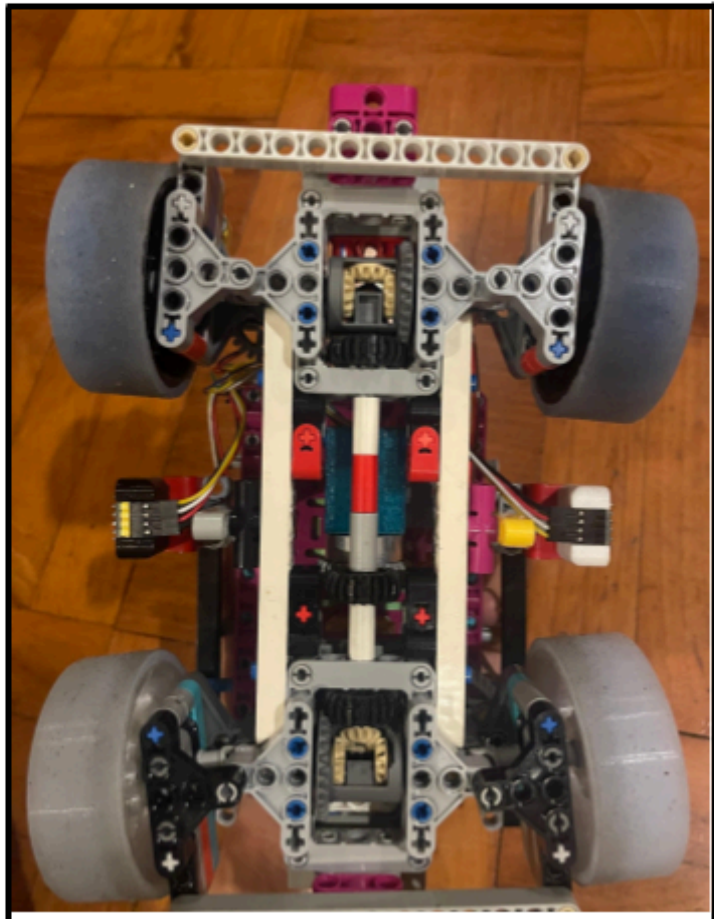
Top View



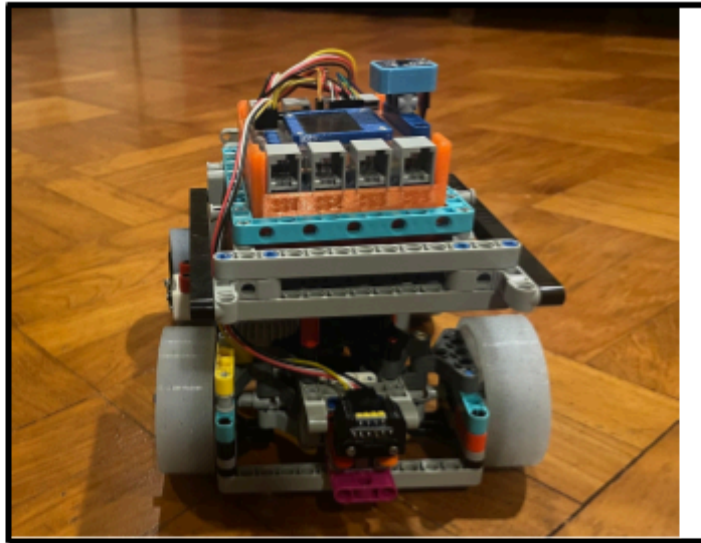
Front View



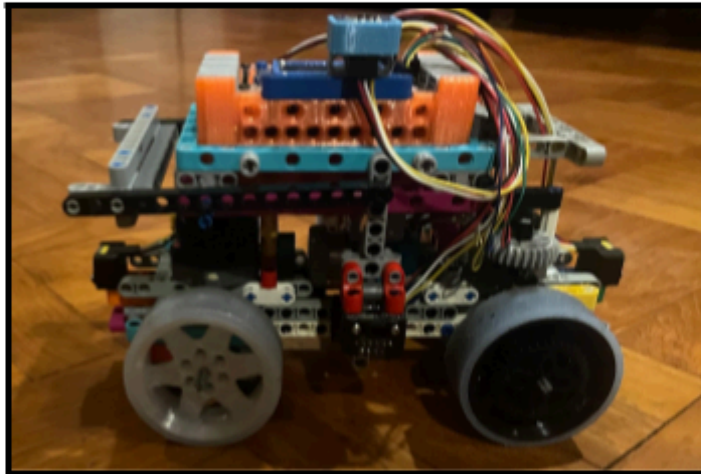
Bottom View



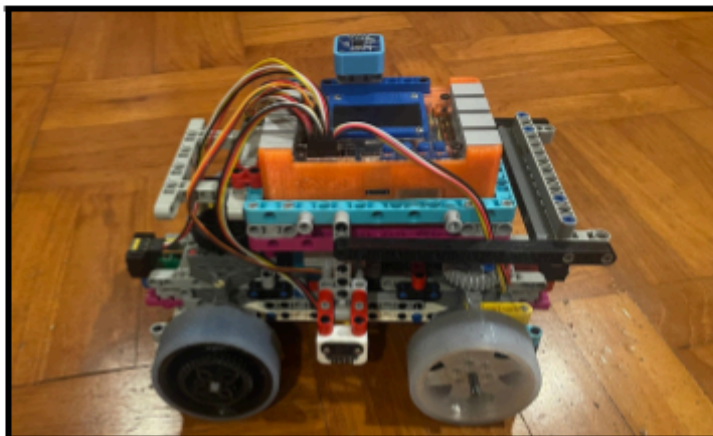
Back View



Right Side View



Left Side View

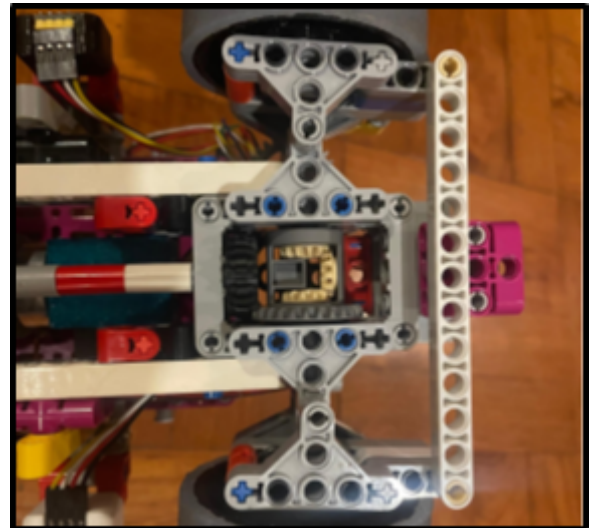


Motors

The motors we have chosen to use as a part of our bot are the EvoMotor300, which has a balance of speed and torque compared to the Evomotor100, which has more torque at the cost of speed. The other reasons we chose this motor were the size limit and the navigation challenge. We felt that having a smaller bot would be easier to navigate through the blocks, so we chose to go for compactness.

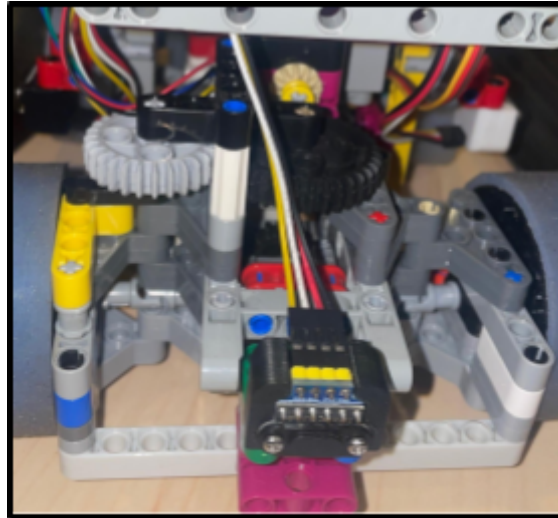
Differential

The drive motor is connected to two differential gears at the bottom of the bot. Having a differential allows the robot's wheel to turn at different speeds, which is crucial to achieving smooth turning.



Steering

For our steering, our team has chosen a geared ackermann steering mechanism, which is a common steering mechanism used by many people in the world, while we also wanted to go for a design of a 4 wheel drive and 4 wheel steering car. This 4 wheel steering system allows us to have a much tighter turning circle, giving us extra manoeuvrability. Moreover, the ackermann steering geometry allows the inner wheels to turn more than the outer ones, providing better and more precise turning. This is due to the inner wheels needing to make a tighter arc than the outer wheels.



Components

Evolution X1

The Evolution X1 Controller Platform is a versatile and powerful development board designed to seamlessly integrate with the LEGO EV3 Mindstorms ecosystem. Built around the robust ESP32-S3 microcontroller, this platform offers a unique combination of connectivity, performance and flexibility. Below are the key features of Exolution X1, and these are some reasons why we chose it.



Key Features:

- **EV3 Mindstorms Compatibility**

The controller is equipped with specialised connectors that enable seamless connection and control of the EV3 Mindstorms motors and sensors. This compatibility allows us to utilise the existing EV3 components while taking advantage of the advanced features of the ESP32-S3. Although we did not use these features in this robot, they are useful to have.

- **Extended GPIO Pinouts**

In addition to EV3 connectors, the platform offers an extensive set of GPIO pinouts. These pinouts allow us to connect various third-party sensors and actuators, broadening the scope for customisation and experimentation.

- **Powerful ESP32-S3 Microcontroller**

The platform is powered by the ESP32-S3 microcontroller, which stands out for its great performance, energy efficiency, and versatile features. It offers dual-core processing, built-in Wi-Fi and Bluetooth, and plenty of peripheral options, making it a flexible and reliable choice for all kinds of applications.

- **User-Friendly Development Environment**

The platform supports popular development environments such as Arduino, MicroPython, and Espressif's IDF, providing a user-friendly and familiar programming experience. It is also backed by comprehensive documentation and strong community support, enhancing the development process.

Time of Flight (ToF)

A Time of Flight (ToF) sensor measures distance by emitting a light signal, typically infrared, and calculating the time it takes for the signal to travel to an object and return. This time is then used to determine the precise distance to the object.

We selected it primarily for its precise distance measurement capabilities. ToF sensors can measure distances with exceptional accuracy—often down to the millimetre—making them ideal for applications that demand precise measurements. They can also measure over a wide range, from just a few centimetres to several metres, depending on the model.

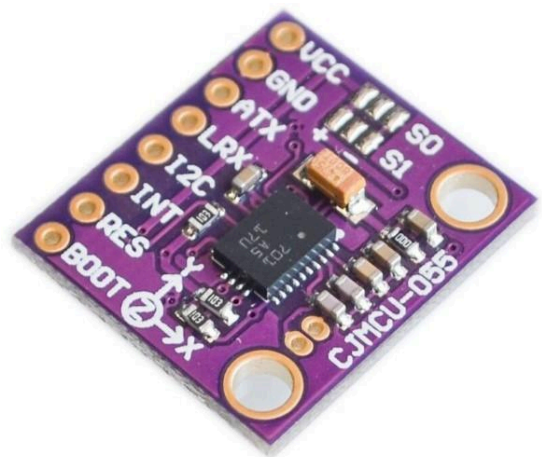
Another advantage is their fast response time. ToF sensors deliver rapid measurements, enabling real-time functions such as gesture recognition, object tracking, and obstacle detection. Since they operate at the speed of light, they can perform distance calculations extremely quickly, providing low-latency data essential in dynamic environments.

Finally, ToF sensors are easy to integrate. They typically use standard communication protocols like I²C or SPI, making them straightforward to connect with microcontrollers and other digital systems. Some models even feature built-in processing, simplifying their use in more complex implementations.

Gyroscope (IMU)

For our gyroscope, we chose the BNO-055. This is a compact 9-axis sensor that combines an accelerometer, gyroscope, magnetometer, and a 32-bit microcontroller with sensor fusion software.

Our main reasons for choosing this component are, firstly, that by integrating an accelerometer, gyroscope, magnetometer, and microcontroller into a single module, it reduces the need for multiple separate sensors, thus reducing the overall data processing requirements and saving memory. Secondly, its compact form also minimises physical space usage, making it ideal for our compactly designed robot.



MaixCam Camera

The MaixCAM is a compact, Linux-based AI vision camera that can run real-time computer-vision models directly on its onboard processor. It supports object detection with models such as YOLOv5, as well as traditional vision functions including colour/blob detection and QR-code recognition. This allows the robot to analyse camera input without relying entirely on the main microcontroller.

We selected the MaixCAM because of its high performing specifications in a small form factor and its ability to perform onboard object detection, which supports obstacle identification and target recognition during the challenges. It has a CPU core of 1GHz and has a NPU that can support up to 1TOPS. Its built-in screen also makes it easier to test the camera feed and verify detections while developing the robot. The compact camera-board form factor further simplifies installation on the robot.

For our project, we trained our own custom vision model using images relevant to our challenge, rather than relying only on the default object categories. The model was then deployed to the MaixCAM, enabling it to recognise the specific objects or visual targets required by our robot. Sipeed supports this workflow through MaixHub or an offline pipeline: prepare and annotate a dataset, train a compatible model, export it to ONNX, convert it to MaixCAM's supported model format, and run it using MaixPy.



Strategy

Preliminary Round Flowchart

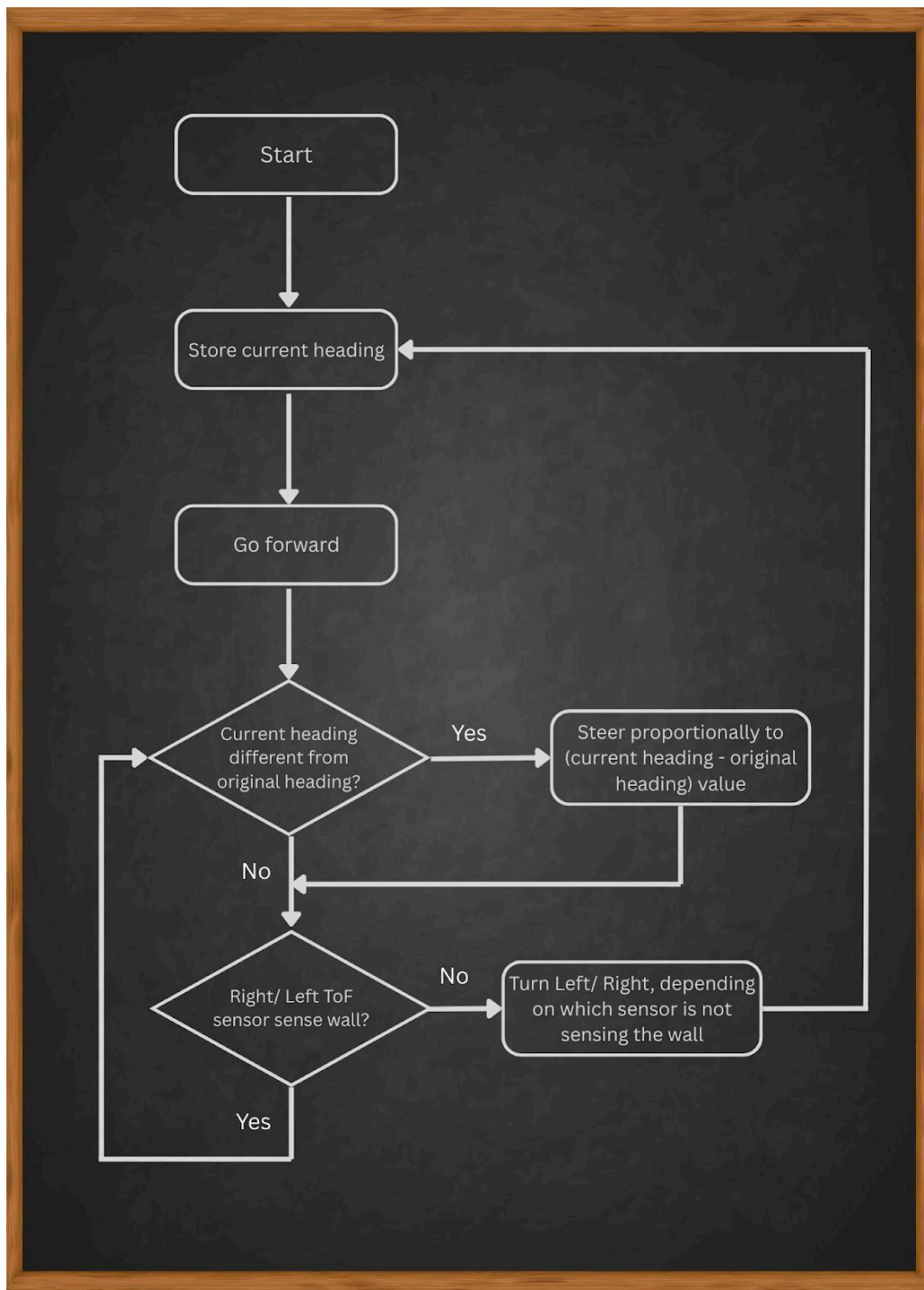


Figure 1

Obstacle Round Flowchart

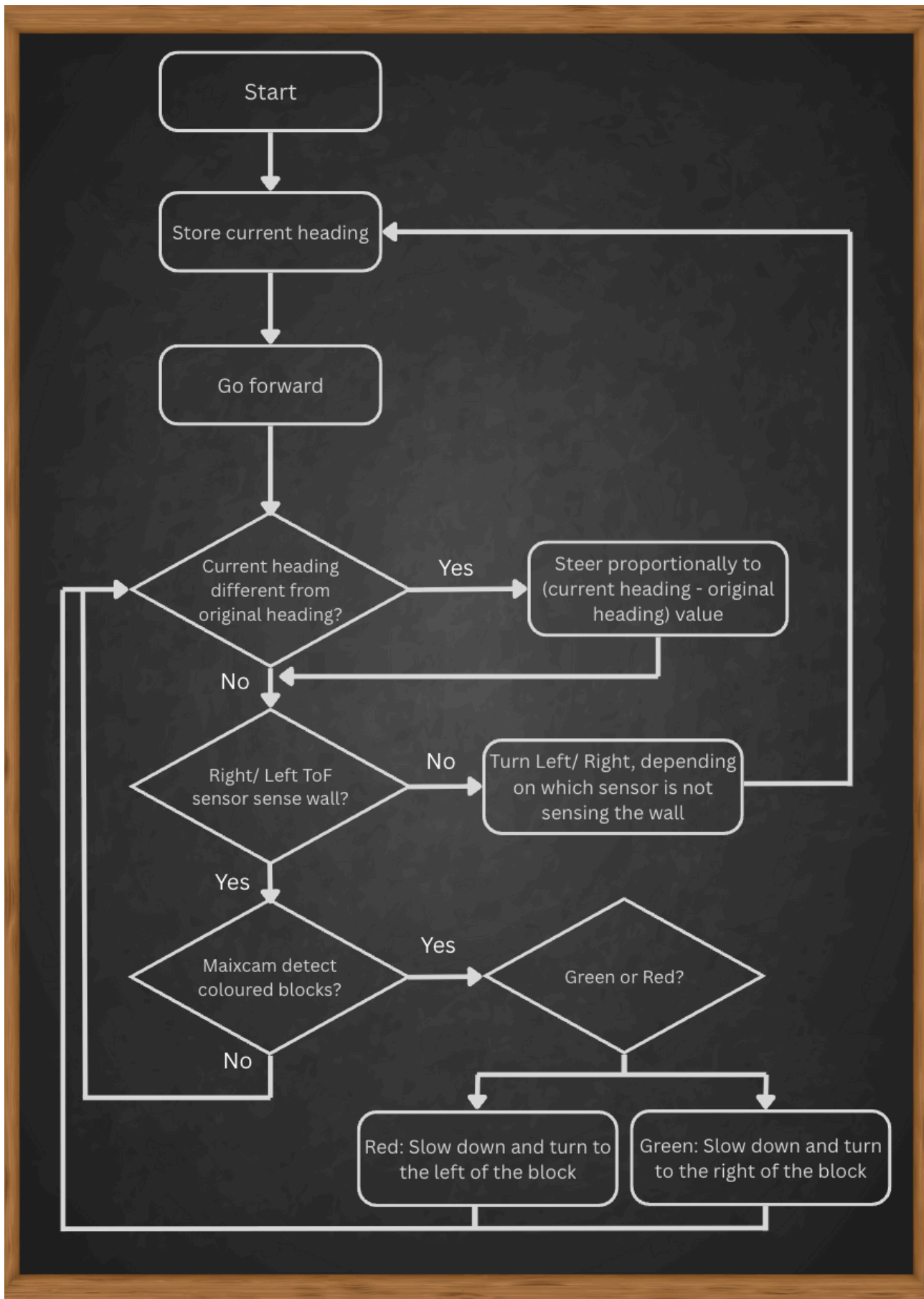


Figure 2

Figure 1 Explanation

During the preliminary rounds, our robot follows a simple flowchart. The first thing the robot does is set the heading when it boots up to be 0° . The robot then drives forward using PD heading tracking. The robot first takes the current heading, and calculates the normalised error from 0° or 360° , and adjusts the steering proportionally. This allows the robot to effectively track the heading, travelling in a perfectly straight line. When the left or right TOFs detect a gap for more than a set value of TOF readings, it will turn in that direction, as the robot knows there is a gap for it to drive into. With this, it completes the circuit.

Figure 2 Explanation

During the preliminary rounds, the robot drives autonomously using four TOF sensors, an IMU, a steering servo, and a drive motor. At startup, it reads its current IMU heading and sets the nearest 45° direction as its target. It then drives forward while using proportional heading control to correct steering errors and maintain a straight path.

The left and right TOF sensors detect gaps in the track. A gap is confirmed only after several consecutive readings above the distance threshold, reducing false detections. The robot turns toward the detected opening, or toward the larger opening if both sides detect a gap. It turns by 90° , then drives forward a fixed encoder distance to clear the junction before searching for the next gap.

The MaixCAM can also detect red and green objects. When a valid object is detected, colour avoidance overrides normal gap navigation. The robot turns 45° away from the object, right for red and left for green, drives past it, turns back toward its original path, and returns to its original heading. This allows the robot to avoid coloured obstacles while still continuing around the course.

Code

In order to make sure everyone on the team could understand the code, we focused on keeping it readable and easy to follow. Even though the code for the self-driving robot is complex, we broke the code into smaller parts and added clear labels.

We kept the code clean by doing the following:

- Wrote comments at important parts of the code to explain what each section does.
- Used libraries and separate classes to keep the main code simple and less crowded.
- Gave variables and functions clear and meaningful names so it's easy to know what they do.

-End-